A framework for estimating the color of a robotic arm

Johanne Johansen 123456789
Peter Peter Petersen 987654321

Masters Project in Engineering (Robot Systems)
Supervisor: Louise Lind
Co-supervisor: Klaus Kristensen

June 1887

The Maersk Mc-Kinney Moeller Institute
University of Southern Denmark

Word count: 1234

Abstract

Many companies are struggling to combine Robotics and Machine Learning...

Contents

1 Introduction

This chapter some basics on blockchain concepts useful to understand this project.

1.1 What is a blockchain?

A blockchain is an immutable ledger that enables the recording of transactions in a P2P network, providing a single source of truth. The first one was invented by Satoshi Nakamoto in 2008 with the release of the Bitcoin whitepaper [5]. It developed a fully peer-to-peer (P2P) network where users can exchange a currency (bitcoin) without third party authorities with the power of rewriting the history of transactions.

A blockchain can be described as a system made of multiple components, the foundamentals are:

- a P2P network of nodes;
- a consensus algorithm;
- a chain of blocks, containing the verified transactions.

There are many variants of blockchains, an important difference to consider to classify them is who can access the network. In a permissionless blockchain (like Ethereum or Bitcoin) anybody can access, anyone can participate in the consensus; permissioned blockchains, on the other hand, restrict the access using a whitelist for trusted peers.

Cryptography primitives

Blockchains rely heavily on cryptographic techniques to ensure security and trust. The main cryptographic tools used are:

Hash functions are one-way mathematical functions that take any input and produce a fixed-size output (called a hash or digest). They are deterministic, meaning the same input always produces the same output. However, it is computationally impossible to reverse the process or find two different inputs that produce the same hash. Bitcoin and Ethereum use SHA-256 and Keccak-256 hash functions respectively.

Digital signatures allow users to prove ownership of their assets and authorize transactions. Each user has a private key (which must be kept secret) and a public key (which can be shared). To send a transaction, a

user signs it with their private key. Anyone can verify the signature using the user's public key, proving that the transaction was authorized by the owner of that private key. Most blockchains use elliptic curve cryptography for digital signatures.

How transactions work

When a user wants to make a transaction on a blockchain, they create a message containing the details (such as the recipient and amount) and sign it with their private key. This signed transaction is then broadcast to the P2P network.

Nodes in the network collect these transactions and group them into blocks. Before a block can be added to the blockchain, it must be validated through the consensus mechanism. Once consensus is reached, the block is added to the chain, and all nodes update their copy of the ledger. This process ensures that all participants have the same view of transaction history.

The chain structure is maintained by including the hash of the previous block in each new block. This creates a tamper-proof link: if someone tries to modify an old transaction, it would change that block's hash, breaking the link to all subsequent blocks. This is why blockchains are described as immutable.

Consensus mechanisms

The consensus mechanism is how the network agrees on which transactions are valid and the order in which they should be recorded. This is a critical challenge in distributed systems, where nodes might be unreliable or malicious [2].

Proof of Work

Bitcoin introduced **Proof of Work** (PoW), where nodes (called miners) compete to solve complex mathematical puzzles. Specifically, miners must find a nonce value that, when combined with the block data and hashed, produces a hash below a certain target threshold. This requires repeated trial and error, making it computationally intensive. The difficulty of the puzzle is adjusted periodically to maintain a consistent block time—approximately 10 minutes per block in Bitcoin, while Ethereum (before its transition to Proof of Stake) targeted roughly 12-15 seconds per block.

The first miner to solve the puzzle gets to add the next block and receives a reward in the form of newly minted cryptocurrency plus transaction fees. This process requires significant computational power and electricity, making it expensive to attack the network. An attacker would need to control more than 50% of the network's total computational power to reliably manipulate the blockchain.

PoW provides strong security through this computational cost, making attacks economically infeasible. It has a proven track record, securing Bitcoin since 2009, and remains permissionless—anyone with computational resources can participate. However, PoW suffers from significant drawbacks. The computational work requires massive amounts of electricity, leading to environmental concerns. Mining also tends to concentrate in areas with cheap electricity, creating centralization risks. Furthermore, the need for specialized hardware (ASICs) creates barriers to entry for individual participants.

Proof of Stake

Proof of Stake (PoS) was developed as an alternative to address PoW's high energy consumption. Instead of computational work, PoS selects validators based on the amount of cryptocurrency they have locked up as collateral (their "stake"). Validators are chosen to propose and validate blocks based on factors such as the size of their stake and how long they've been staking. When selected, a validator creates a new block and other validators attest to its validity. Validators who act dishonestly risk losing their stake through a process called "slashing," creating an economic incentive for honest behavior.

The fundamental differences between PoW and PoS reflect different security models. PoS requires capital (staked tokens) rather than computational power, making it dramatically more energy efficient—consuming only a fraction of the energy compared to PoW. The attack model also differs: in PoS, attacking the network requires acquiring and risking a significant stake, whereas PoW requires controlling computational resources. Block production is deterministic in PoS, with validators selected according to protocol rules, rather than the competitive race to solve puzzles characteristic of PoW.

PoS offers several advantages, particularly its minimal computational requirements, which eliminate the need for specialized hardware and lower barriers to entry. The economic security model creates strong disincentives for attacks, as malicious behavior results in financial penalties. However, PoS faces its own challenges. The "nothing at stake" problem suggests that validators could theoretically validate multiple competing chains without cost. There are also concerns about wealth concentration, as those with more tokens have more influence over the network. Finally, PoS is newer and less battle-tested than PoW, having less operational history in securing high-value networks.

1.2 Ethereum and smart contracts

While Bitcoin was designed primarily as a digital currency, Ethereum extended the blockchain concept to support general-purpose computation [2]. Launched in 2015 by Vitalik Buterin and others, Ethereum introduced **smart contracts**:

programs that run on the blockchain and automatically execute when certain conditions are met.

Ethereum consensus mechanism

Ethereum initially used Proof of Work with the Ethash algorithm, which was designed to be ASIC-resistant and more memory-intensive than Bitcoin's SHA-256. However, the Ethereum community recognized the limitations of PoW, particularly its energy consumption and scalability constraints.

In September 2022, Ethereum underwent one of the most significant upgrades in blockchain history, known as **The Merge**. This event transitioned Ethereum from Proof of Work to Proof of Stake, implementing a consensus mechanism called **Gasper**, which combines the Casper FFG (Friendly Finality Gadget) and LMD GHOST (Latest Message Driven Greediest Heaviest Observed SubTree) protocols.

In Ethereum's PoS system, validators must stake 32 ETH to participate in block production and validation. These validators are randomly selected to propose blocks in assigned time slots every 12 seconds, while other validators attest to the validity of proposed blocks. Validators earn rewards for honest participation and face penalties (slashing) for malicious behavior. Finality is achieved through a checkpoint system, where blocks become irreversible after being finalized. This transition reduced Ethereum's energy consumption by approximately 99.95% while maintaining security and enabling future scalability improvements.

Smart contracts and the EVM

A smart contract is a program written in a high-level language (such as Solidity or Vyper) and deployed to the blockchain. Once deployed, the contract's code cannot be changed, and it will execute exactly as programmed whenever someone interacts with it. This immutability enables the creation of trustless decentralized applications (DApps) that can operate without central control.

Smart contracts are executed by the **Ethereum Virtual Machine** (EVM), a quasi-Turing-complete computational environment that runs on every node in the network. The EVM is a stack-based virtual machine with a word size of 256 bits, designed to execute bytecode compiled from high-level smart contract languages. When a transaction calls a smart contract, every node executes the same code and updates their state accordingly, ensuring that all nodes maintain consensus on the state of all accounts and contracts.

The EVM maintains several types of state: **account state** (balances and nonces for externally owned accounts), **contract state** (storage, code, and account balance for smart contracts), and **transaction state** (temporary data during transaction execution).

Opcodes

At the lowest level, the EVM executes **opcodes**: simple operations that form the instruction set of the virtual machine. Smart contract bytecode consists of a sequence of these opcodes, each performing a specific operation. The EVM instruction set includes arithmetic operations (ADD, SUB, MUL, DIV, MOD), comparison operations (LT, GT, EQ, ISZERO), bitwise operations (AND, OR, XOR, NOT, BYTE), and stack operations (PUSH, POP, DUP, SWAP). Memory and storage are manipulated through dedicated opcodes—MLOAD and MSTORE for temporary memory, while SLOAD and SSTORE handle persistent contract storage. Control flow is managed by jump instructions (JUMP, JUMPI, JUMPDEST), and the EVM provides access to blockchain context through opcodes like ADDRESS, BALANCE, CALLER, CALLVALUE, TIMESTAMP, and NUMBER. Finally, execution opcodes like CALL, DELEGATECALL, STATICCALL, CREATE, and SELFDESTRUCT enable contract interaction and deployment.

Each opcode has an associated gas cost that reflects its computational complexity and resource usage. For example, simple arithmetic operations cost 3 gas, while writing to storage (SSTORE) costs significantly more (20,000 gas for setting a new value) because it requires updating the permanent blockchain state.

Gas mechanism

To prevent infinite loops and abuse, Ethereum uses a concept called **gas**. Gas serves multiple critical purposes in the Ethereum ecosystem. First, it provides **resource metering**, ensuring every computation on the EVM costs a certain amount of gas proportional to the computational resources required. This metering prevents denial-of-service attacks by requiring payment for computation, making it economically infeasible for attackers to overwhelm the network with expensive operations. Gas also serves as **validator compensation**—users pay for the gas their transactions consume, compensating validators for processing transactions and storing state. Finally, gas creates a **priority mechanism** where users can offer higher gas prices to incentivize validators to include their transactions faster.

The gas system operates through a straightforward model. Each operation has a fixed **gas cost** measured in gas units. Users specify a **gas limit** (the maximum gas they're willing to consume) and a **gas price** or max fee (the amount of ETH they'll pay per unit of gas). The total transaction cost equals gas used multiplied by gas price. If a transaction runs out of gas before completing, it reverts, but the gas spent up to that point is still consumed as compensation for the computational work performed. Unused gas is refunded to the sender.

Since the London hard fork (EIP-1559), Ethereum uses a more sophisti-

cated fee market with a **base fee** that is automatically adjusted based on network congestion and burned (removed from circulation), plus an optional **priority fee** (tip) that goes to validators. This creates more predictable transaction fees and introduces a deflationary mechanism for ETH.

Different types of operations have vastly different gas costs reflecting their resource requirements. Simple arithmetic operations cost only 3-5 gas, while SHA3 hashing requires 30 gas plus 6 gas per word. Storage operations are particularly expensive: reading from storage (SLOAD) costs 2,100 gas, while writing to storage (SSTORE) costs 20,000 gas for setting a new value or 5,000 gas for modifying an existing one. Contract creation requires 32,000 gas plus the deployment cost, and every transaction has a base cost of 21,000 gas. This pricing structure incentivizes developers to write efficient code and minimize on-chain storage usage, as storage operations are among the most expensive.

Decentralized applications

Smart contracts enable a wide range of applications beyond simple payments. Some examples include:

- **Decentralized Finance (DeFi)**: Financial services like lending, borrowing, and trading without traditional intermediaries.
- **Non-Fungible Tokens (NFTs)**: Unique digital assets that represent ownership of items like art, collectibles, or real-world assets.
- **Decentralized Autonomous Organizations (DAOs)**: Organizations governed by smart contracts where decisions are made through voting by token holders.
- **Supply chain tracking**: Recording the movement of goods through a supply chain with transparency and immutability.

The key advantage of these applications is that they inherit the blockchain's properties: they are transparent, censorship-resistant, and don't require trust in a central authority. However, they also face challenges such as scalability, high transaction costs, and the irreversibility of bugs in smart contract code.

2 Background

This chapter introduces the key technologies and concepts that form the foundation of this project: decentralized finance, the AAVE lending protocol, and modular smart accounts.

2.1 Decentralized Finance

Decentralized Finance, commonly known as DeFi, refers to financial services built on blockchain technology that operate without traditional intermediaries like banks or brokers. Instead of relying on centralized institutions, DeFi applications use smart contracts to automate financial operations, making them transparent, permissionless, and accessible to anyone with an internet connection.

Core concepts

Traditional finance relies on trusted intermediaries to facilitate transactions. When you deposit money in a bank, the bank holds your funds and decides how to use them. When you take out a loan, the bank evaluates your creditworthiness and sets the terms. DeFi replaces these intermediaries with code: smart contracts that execute automatically according to predefined rules.

The key principles of DeFi include:

- **Permissionless access**: Anyone can use DeFi services without needing approval from a central authority. There are no credit checks, identity verification, or geographic restrictions.
- **Transparency**: All transactions and smart contract code are publicly visible on the blockchain. Users can verify exactly how protocols work and audit their security.
- **Composability**: DeFi protocols can interact with each other, allowing developers to combine different services like building blocks. This is often called "money legos."
- **Self-custody**: Users maintain control of their own assets through their private keys, rather than trusting a third party to hold their funds.

Token standards

Tokens are digital assets that exist on a blockchain. On Ethereum, the most common token standard is **ERC-20**, which defines a common interface for fungible tokens. Fungible means that each token is identical and interchangeable with any other token of the same type, just like how one euro is equivalent to any other euro.

The ERC-20 standard specifies functions that all compliant tokens must implement:

- `balanceOf(address)`: Returns how many tokens an address owns.
- `transfer(address, amount)`: Sends tokens to another address.
- `approve(address, amount)`: Allows another address (typically a smart contract) to spend tokens on your behalf.
- `transferFrom(from, to, amount)`: Transfers tokens from one address to another, used by approved spenders.

This standardization is crucial for DeFi because it allows protocols to work with any ERC-20 token without needing custom code for each one. A lending protocol can accept any ERC-20 token as collateral, and a decentralized exchange can trade any pair of ERC-20 tokens.

Lending and borrowing

One of the most fundamental DeFi services is lending and borrowing. In traditional finance, if you want to earn interest on your savings, you deposit money in a bank. If you need a loan, you apply to the bank and provide collateral or proof of income.

DeFi lending protocols work differently. Users who want to earn interest deposit their tokens into a **liquidity pool**, a smart contract that holds funds from many different users. These pooled funds are then available for other users to borrow.

Borrowers must provide **collateral** to take out a loan. Because DeFi is permissionless and pseudonymous, there is no way to enforce repayment through legal means. Instead, borrowers must lock up assets worth more than what they borrow. If the value of their collateral drops below a certain threshold (the **liquidation threshold**), anyone can repay part of the loan and claim the collateral at a discount. This mechanism ensures that lenders are always protected, even if borrowers default.

Interest rates in DeFi lending are typically determined algorithmically based on supply and demand. When there is high demand for borrowing a particular asset, interest rates increase to attract more lenders. When demand is low, rates decrease.

2.2 AAVE Protocol

AAVE is one of the largest and most widely used DeFi lending protocols [1]. Originally launched as ETHLend in 2017, it was rebranded to AAVE (Finnish for "ghost") in 2020 with the release of AAVE V2. The protocol has since evolved to AAVE V3, which introduced new features for capital efficiency and risk management.

How AAVE works

AAVE operates as a system of liquidity pools, with one pool for each supported asset. Users interact with the protocol through a central `Pool` contract that manages all lending and borrowing operations.

Supplying assets: When users supply assets to AAVE, they deposit tokens into the protocol and receive **aTokens** in return. These aTokens represent the user's share of the liquidity pool and automatically accrue interest over time. For example, if you supply 100 DAI, you receive 100 aDAI. As borrowers pay interest, the balance of aDAI in your wallet increases. When you want to withdraw, you redeem your aTokens for the underlying asset plus accumulated interest.

Borrowing assets: To borrow from AAVE, users must first supply collateral. The amount they can borrow depends on the **Loan-to-Value (LTV)** ratio of their collateral. For example, if an asset has an LTV of 80%, a user who supplies $1000 worth of that asset can borrow up to $800. AAVE supports two types of interest rates: **stable rates** that remain relatively constant, and **variable rates** that fluctuate based on market conditions.

Liquidations: If a borrower's collateral value drops and their position becomes undercollateralized (meaning their debt exceeds the allowed threshold), liquidators can repay part of the debt and receive the collateral at a discount. This mechanism protects lenders and maintains the protocol's solvency.

AAVE V3 features

AAVE V3 introduced several improvements over previous versions:

- **Efficiency Mode (E-Mode)**: Allows higher borrowing power when supplying and borrowing correlated assets, such as different stablecoins.
- **Isolation Mode**: Limits exposure to riskier assets by capping how much can be borrowed against them.
- **Portal**: Enables seamless transfer of liquidity between AAVE deployments on different networks.
- **Gas optimizations**: Reduced transaction costs through more efficient smart contract design.

AAVE smart contracts

The AAVE protocol consists of several key smart contracts:

- `Pool`: The main entry point for users. Handles supply, borrow, repay, and withdraw operations.
- `PoolAddressesProvider`: A registry that stores the addresses of all protocol contracts, making it easier to interact with the protocol.
- `AToken`: The interest-bearing token that represents supplied assets.
- `VariableDebtToken` and `StableDebtToken`: Tokens that represent borrowed amounts with variable or stable interest rates.
- `Oracle`: Provides price feeds for assets, used to calculate collateral values and determine liquidations.

To interact with AAVE programmatically, external contracts call functions on the `Pool` contract. The most commonly used functions include:

- `supply(asset, amount, onBehalfOf, referralCode)`: Deposits tokens into the protocol.
- `borrow(asset, amount, interestRateMode, referralCode, onBehalfOf)`: Borrows tokens against supplied collateral.
- `repay(asset, amount, interestRateMode, onBehalfOf)`: Repays borrowed tokens.
- `withdraw(asset, amount, to)`: Withdraws supplied tokens from the protocol.

2.3 Account Abstraction

Traditional Ethereum accounts, known as Externally Owned Accounts (EOAs), are controlled by private keys. Every transaction must be signed by the private key, and the account has limited functionality: it can only send transactions and hold assets. This creates several problems:

- **Key management**: Losing a private key means losing access to all funds permanently.
- **No recovery options**: There is no way to recover an EOA if the key is lost or compromised.
- **Gas payment**: Users must always have ETH to pay for gas, even if they only want to interact with other tokens.

- **Limited automation**: EOAs cannot execute transactions automatically or set complex conditions.

Account Abstraction addresses these limitations by replacing EOAs with smart contract accounts, often called **Smart Accounts**. These accounts are smart contracts that can implement custom logic for authentication, recovery, and transaction execution.

ERC-4337

ERC-4337 is a standard that brings account abstraction to Ethereum without requiring changes to the protocol itself [3]. It introduces a new transaction flow:

1. Users create **UserOperations** instead of regular transactions. These describe what action the user wants to perform.
2. UserOperations are sent to a separate **mempool** (a waiting area for pending operations) rather than the regular transaction mempool.
3. **Bundlers** collect UserOperations and submit them to the blockchain as regular transactions.
4. A singleton contract called the **EntryPoint** processes the bundled operations and calls the target smart accounts.
5. Each smart account validates the operation according to its own rules and executes the requested action.

This architecture enables features that were previously impossible:

- **Custom authentication**: Smart accounts can use any validation logic, not just ECDSA signatures. This enables multi-signature wallets, social recovery, biometric authentication, and more.
- **Sponsored transactions**: A third party (called a **Paymaster**) can pay for gas on behalf of users, enabling gasless transactions.
- **Batched transactions**: Multiple operations can be combined into a single transaction, saving gas and improving user experience.

ERC-7579: Modular Smart Accounts

While ERC-4337 defines how smart accounts interact with the network, it does not specify how accounts should be structured internally. Different wallet providers implemented their own architectures, leading to fragmentation in the ecosystem.

ERC-7579 addresses this by defining a standard interface for **modular smart accounts** [6]. Instead of building monolithic smart accounts with all features hardcoded, ERC-7579 enables accounts to install and uninstall modules that add specific functionality.

The standard defines four types of modules:

- **Validators**: Modules that verify whether an operation is authorized. They can implement different authentication schemes like ECDSA signatures, multi-sig, passkeys, or social recovery.
- **Executors**: Modules that can trigger actions on behalf of the account. They enable automation, scheduled transactions, and integration with external protocols.
- **Hooks**: Modules that run before and/or after execution. They can enforce policies, implement spending limits, or add additional security checks.
- **Fallback handlers**: Modules that handle calls to functions not defined on the account itself, enabling extensibility.

Each module type has a specific interface it must implement. For example, all modules must implement:

- `onInstall(bytes data)`: Called when the module is installed on an account.
- `onUninstall(bytes data)`: Called when the module is removed from an account.
- `isModuleType(uint256 typeID)`: Returns whether the module is of a specific type.

Executor modules

Executor modules are particularly relevant for DeFi automation. They allow external contracts or accounts to trigger actions through the smart account, following predefined rules. Common use cases include:

- **Automated DeFi strategies**: Executors can automatically rebalance positions, harvest yields, or manage liquidity based on market conditions.
- **Scheduled transactions**: Recurring payments or periodic operations can be triggered by executors.
- **Protocol integrations**: Executors can provide simplified interfaces for interacting with complex DeFi protocols.

An executor module can call the smart account's `execute` function to perform actions like token transfers, contract calls, or multi-step operations. The modular architecture ensures that these actions are performed with the account's permissions while maintaining security through the account's validation logic.

This combination of account abstraction and modular architecture creates a powerful foundation for building sophisticated DeFi applications that can interact with protocols like AAVE in automated and user-friendly ways.

3 Analysis and Design

We give an example of a bibliographic reference [4].

... We summarize something in Table 3.1.

LaTeX indents a new paragraph unless you specify `\noindent`.

We can also refer to Figure 3.1.

Feature	Benefit
Arms	Can move external things
Legs	Code move platforms

Table 3.1: Advantages of Robots

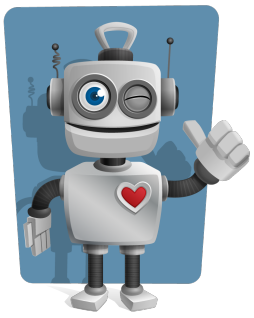

Figure 3.1: A good looking robot

4 Implementation

We can refer to Chapter 3 for something.

Here is an example formula to compute a sum:

$$\sum_{i=1}^{n} = \frac{n(n+1)}{2}$$

Math can also be included inline $a(b+c) = ab + ac$ in a sentence.

We refer to Listing 4.1 for the Java implementation.
Code snippets like `System.out.println` can also be given inside a sentence.

Finally we can refer to some material in Appendix A.

```
class Sum {
  public static void main(String[] args) {
    int n = 5;    // the input
    int sum = n * (n+1);
    System.out.println("The sum is: " + sum);
  }
}
```

Listing 4.1: Our sum implementation

5 Conclusion

Overall our project was a success.

A Appendix

We include here the API documentation for our library.

Bibliography

[1] Aave. Aave v3 technical paper, 2022.

[2] Andreas M. Antonopoulos and Gavin Wood. *Mastering Ethereum: Building Smart Contracts and DApps*. O'Reilly Media, Sebastopol, CA, 2018.

[3] Vitalik Buterin, Yoav Weiss, Dror Tirosh, Shahaf Nacson, Alex Forshtat, Kristof Gazso, and Tjaden Hess. Erc-4337: Account abstraction using alt mempool, 2021.

[4] Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein. *Introduction to Algorithms*. The MIT Press, Cambridge, Massachusetts, second edition, 2001.

[5] Satoshi Nakamoto. Bitcoin: A peer-to-peer electronic cash system, 2008.

[6] zeroknots, Konrad Kopp, Taek Lee, Fil Makarov, Elim Poon, and Lyu Min. Erc-7579: Minimal modular smart accounts, 2023.